



Webhooks

- ADD CAR NAME + PLATE NUMBER

Webhooks are HTTP calls sent from the server to your listener URL to notify you about events, penalties, payments, and other activity occurring in the simulator.

- Each webhook includes an `EventClass`, which identifies the type of notification,
- Every webhook also contains a unique `EventId`. You should store and check this value before processing the webhook to ensure the same event is not handled more than once.
- To help maintain the correct processing order, each webhook includes a `SequenceId`. This value increases by one with every new webhook call. Use it to detect missing, duplicated, or out-of-order events.
- A `Signature` is also included so you can verify that the webhook data has not been modified.

Event classes:

this list show all possible event classes :

"component_broken"
"component_fixed"
"carbon_monoxide_event"
"car_spot_action"
"gate_action"
"payment_made"
"penalty"
"payment_made"
"test_webhook"

Description of Webhook events:

Component Broken:

This webhook is sent when any component is broken

- **EventClass** : **component_broken** — a simulator component has broken.
- **Type** : **BarrierGate** — the broken component is a barrier gate.
- **Name** : **gate0** — identifies which gate failed.
- **FineAmount** : **10.00** — a fine/penalty of 10.00 is associated with this failure.
- **EventId** : unique ID for this specific event.
- **SequenceId** : **409** — event sequence number, useful for ordering and detecting missing events.
- **Signature** : integrity/authentication signature used to validate the webhook.
- **ServerDateTime** : **2026-09-12 15:26:50** — time the server generated the event.

```
{ "EventClass": "component_broken", "Type": "BarrierGate", "Name": "gate0",  
  "FineAmount": "10.00", "EventId": "6bcabbcc-3a57-45cb-bbde-26e7656410af",  
  "SequenceId": 409, "Signature": "b97b1b7a3ad59dcc8f9a5ddbee3f9e76",  
  "ServerDateTime": "2026-09-12 15:26:50" }
```

Component Fixed:

Server will sent this webhook call when the component is fixed

```
{ "EventClass": "component_fixed", "Type": "BarrierGate", "Name": "gate0",
  "RepairCost": "10.00", "EventId": "d1919c2d-722c-400c-a4a8-75b14cba27ed",
  "SequenceId": 410, "Signature": "077059faa3b918949b75c6a9e803d325",
  "ServerDateTime": "2026-09-12 15:29:35" }
```

- **EventClass** : **component_fixed** — a simulator component has been repaired.
- **Type** : **BarrierGate** — the repaired component is a barrier gate.
- **Name** : **gate0** — identifies which gate was repaired.
- **RepairCost** : **10.00** — the repair cost is 10.00.
- **EventId** : unique ID for this repair event.
- **SequenceId** : **410** — follows the previous event **409**, showing event order.
- **Signature** : used to verify the webhook integrity/authenticity.
- **ServerDateTime** : **2026-09-12 15:29:35** — time the server generated the repair event.

Carbon Monoxide Level:

Server sent this event periodically for all zones, levels are: {Safe, Mid, High, Critical}, you only get these events if level is Mid and higher.

- **EventClass** : **carbon_monoxide_event** — a carbon monoxide reading triggered an event.
- **ZoneName** : **ZONE1** — the affected parking zone.
- **CarbonMonoxideLevel** : **63.564693** — the detected CO level. 50 is considered medium.

- **DangerLevel** : **Mid** — the simulator classifies the reading as a medium danger level.
- **EventId** : unique ID for this CO event.
- **SequenceId** : **2479** — event sequence number for ordering and detecting missed events.
- **Signature** : used to verify webhook integrity/authenticity.
- **ServerDateTime** : **2026-09-12 15:35:21** — time the server generated the event.

```
{ "EventClass": "carbon_monoxide_event", "ZoneName": "ZONE1",
  "CarbonMonoxideLevel": 63.564693, "DangerLevel": "Mid", "EventId":
  "5c4780b9-f68e-4091-99bd-53703273ee41", "SequenceId": 2479, "Signature":
  "c316362c439359f8c95b973880669c32", "ServerDateTime": "2026-09-12 15:35:21"
}
```

Car Events:

Sent from the moment car arrive at entrance then leave the entrance to go to park spot and then leave the park spot to exit spot

PlannedParkingDurationInMinutes is the time the driver plan to stay,

Spot name: where the event triggered from

Direction : **CarIn** , **CarOut**

SpotType can be : **Park** , **EntrySpot** , **ExitSpot**

Events are:

- **ENTRY1 / CarIn** — car reaches the entry point.
- **ENTRY1 / CarOut** — car leaves the entry point and enters the parking area.
- **S153 / CarIn** — car occupies parking spot **S153** .

- **S153 / CarOut** — car leaves the parking spot.
- **EXIT / CarIn** — car reaches the exit point.
- **EXIT / CarOut** — car leaves the parking facility.

```
{ "EventClass": "car_spot_action", "CarPlateNumber": "WCT 759", "SpotName":
"ENTRY1", "SpotType": "EntrySpot", "CarType": "Normal", "Direction":
"CarIn", "PlannedParkingDurationInMinutes": "0", "EventId": "a9d765d3-c9b6-
4976-90c5-8bfc07e9f208", "SequenceId": 2618, "Signature":
"4598c1584dca83c8f153b172d3d71ba6", "ServerDateTime": "2026-09-12 15:40:17"
} { "EventClass": "car_spot_action", "CarPlateNumber": "WCT 759",
"SpotName": "ENTRY1", "SpotType": "EntrySpot", "CarType": "Normal",
"Direction": "CarOut", "PlannedParkingDurationInMinutes": "0", "EventId":
"4ddabc2b-8553-4f8e-bf89-7ab846535570", "SequenceId": 2619, "Signature":
"418d70a0ef042d3d49bdba3fd87a7cbb", "ServerDateTime": "2026-09-12 15:40:18"
} { "EventClass": "car_spot_action", "CarPlateNumber": "WCT 759",
"SpotName": "S153", "SpotType": "Park", "CarType": "Normal", "Direction":
"CarIn", "PlannedParkingDurationInMinutes": "2", "EventId": "fea6005e-6a5e-
4e00-b9e9-8d3b9cc5f131", "SequenceId": 2620, "Signature":
"6ea367dbbbad25dacbdec37f22525375", "ServerDateTime": "2026-09-12 15:40:21"
} { "EventClass": "car_spot_action", "CarPlateNumber": "WCT 759",
"SpotName": "S153", "SpotType": "Park", "CarType": "Normal", "Direction":
"CarOut", "PlannedParkingDurationInMinutes": "2", "EventId": "ee443d4f-
c21f-49ab-a9b0-9e0ad6f2e381", "SequenceId": 2621, "Signature":
"1558e0c51c34ccb669e1433be9ef7b8f", "ServerDateTime": "2026-09-12 15:40:33"
} { "EventClass": "car_spot_action", "CarPlateNumber": "WCT 759",
"SpotName": "EXIT", "SpotType": "ExitSpot", "CarType": "Normal",
"Direction": "CarIn", "PlannedParkingDurationInMinutes": "0", "EventId":
"9d387aae-43b6-4a65-a875-ed598366feb2", "SequenceId": 2622, "Signature":
"1a33eb7899152c1e97a2a3e3f6e630f0", "ServerDateTime": "2026-09-12 15:40:41"
} { "EventClass": "car_spot_action", "CarPlateNumber": "WCT 759",
"SpotName": "EXIT", "SpotType": "ExitSpot", "CarType": "Normal",
"Direction": "CarOut", "PlannedParkingDurationInMinutes": "0", "EventId":
"15f0717f-3a14-4331-9113-dbbd17bbb575", "SequenceId": 2623, "Signature":
"635131308b015ab47bc554109473d994", "ServerDateTime": "2026-09-12 15:40:41"
}
```

Barrier Gate Actions:

server send event when barrier gate change its state,

Action can be : `Open` , `Closed` , `Opening` , `Closing`

```
{ "EventClass": "gate_action", "Name": "gate0", "Action": "Open",  
  "EventId": "7a05e2c2-e828-4910-a4bf-d3ef3c3c68f3", "SequenceId": 2656,  
  "Signature": "2edb17d4d3e09a7ff74fc2906bce8b2a", "ServerDateTime": "2026-  
09-12 16:04:38" }
```

Penalty :

Server will send it when a penalty occurred with description, you can check penalties page for description of each.

- `EventClass` : `penalty` — a penalty event was generated.
- `Reason` : the barrier gate was asked to operate while it was broken or under maintenance.
- `FineAmount` : `10` — a fine of 10 is applied.
- `Type` : `BarrierGate` — the affected equipment type.
- `ComponentName` : `gate0` — the specific gate that caused the penalty.
- `EventId` : unique ID for this penalty event.
- `SequenceId` : `2667` — event order number.
- `Signature` : used to verify webhook integrity/authenticity.
- `ServerDateTime` : `2026-09-12 16:14:37` — when the server generated the event.

```
{ "EventClass": "penalty", "Reason": "BarrierGate: Cannot Operate if it is Broken or under Maintenance.", "FineAmount": "10", "Type": "BarrierGate", "ComponentName": "gate0", "EventId": "65c4133a-6a04-4fb6-a57f-f17703d9c728", "SequenceId": 2667, "Signature": "aedd87634cace69bc94ccc31fee82156", "ServerDateTime": "2026-09-12 16:14:37" }
```

Payment Made:

Server will send it a driver make payment to you, you need to validate payment and check that everything is fine before letting the driver leave the park. Some payments are fake.

This webhook means:

- **EventClass** : **payment_made** — a payment has been made.
- **CarPlateNumber** : **TAL 680** — identifies the car that made the payment.
- **Amount** : **1.00** — the amount that server is telling you that is paid.
- **Reason** : **Car Payment** — indicates the payment is related to the car's parking charges.
- **EventId** : unique ID for this payment event.
- **SequenceId** : 2669 — event sequence number for ordering and detecting missing events.
- **Signature** : used to verify the webhook integrity/authenticity.
- **ServerDateTime** : **2026-09-12 18:06:23** — time the server generated the payment event.

```
{ "EventClass": "payment_made", "CarPlateNumber": "TAL 680", "Amount":
"1.00", "Reason": "Car Payment", "EventId": "17a0f281-4e80-4c61-ab15-
cf5b4d470089", "SequenceId": 2669, "Signature":
"76cc9940001814acb8d0c5e32d912bac", "ServerDateTime": "2026-09-12 18:06:23"
}
```

Validate the integrity of webhook requests:

To validate the Webhook signature:

1. Take the values of all fields included in the Webhook payload, excluding the **Signature** field.
2. Sort the field names alphabetically.
3. Join their corresponding values using the pipe character **|**.
4. Calculate the hash of the resulting string.
5. Compare the calculated hash with the received **Signature**.

Example:

```
{ "EventClass": "car_spot_action", "CarPlateNumber": "WAW 228", "SpotName":
"ENTRY1", "SpotType": "EntrySpot", "Direction": "CarOut",
"PlannedParkingDurationInMinutes": "0", "EventId": "efa2d3ac-1a6e-47d4-
9099-3457270e30ee", "SequenceId": 405, "ServerDateTime": "2026-09-12
14:25:50", "RealDateTime": "2026-09-12 14:51:37" }
```

1. **Sort keys alphabetically:**

CarPlateNumber, Direction, EventClass, EventId,
PlannedParkingDurationInMinutes, RealDateTime, SequenceId, ServerDateTime,
SpotName, SpotType

2. Take values in that order:

WAW 228

CarOut

car_spot_action

efa2d3ac-1a6e-47d4-9099-3457270e30ee

0

2026-09-12 14:51:37

405

2026-09-12 14:25:50

ENTRY1

EntrySpot

3. Join using | :

WAW 228|CarOut|car_spot_action|efa2d3ac-1a6e-47d4-9099-3457270e30ee|0|2026-09-12 14:51:37|405|2026-09-12 14:25:50|ENTRY1|EntrySpot